



۸۱۰۱۰۱۴۰۸

۸۱۰۱۰۱۳۹۲

بخش اول: آشنایی با RPC و فناوری‌های مختلف آن

نام و نام خانوادگی اعضای گروه:

بابک حسینی محتشم

بهراد بینایی



تمرین شماره ۲

۱ تعریف RPC

RPC مخفف Remote Procedure Call است؛ مکانیزمی که به یک برنامه اجازه می‌دهد تابعی را که روی ماشین یا پردازش دیگری در شبکه قرار دارد، فراخوانی کند. درست به همان شکلی که یک تابع محلی را صدا می‌زند. فراخواننده نیازی ندارد از جزئیات شبکه، پروتکل ارتباطی، یا نحوه اجرای تابع در سمت راه دور آگاه باشد. مفهوم RPC نخستین بار در دهه ۱۹۸۰ توسط Birrell و Nelson در مقاله‌ای تأثیرگذار معرفی شد و از آن زمان پایه بسیاری از سیستم‌های توزیع شده بوده است.

۲ ایده اصلی فراخوانی تابع از راه دور

ایده مرکزی RPC این است که پیچیدگی ارتباط شبکه را از دید برنامه‌نویس پنهان کند. در یک فراخوانی RPC مراحل زیر به ترتیب رخ می‌دهند:

۱. برنامه سمت کلاینت، تابع مورد نظر را با آرگومان‌های مشخص فراخوانی می‌کند.
۲. لایه RPC آرگومان‌ها را بسته‌بندی (marshal/serialize) کرده و درخواست را به شبکه می‌فرستد.
۳. سمت سرور درخواست را دریافت، از بسته‌بندی خارج (unmarshal) کرده، تابع واقعی را اجرا می‌کند.
۴. نتیجه به همین ترتیب سریالایز شده و برگشت داده می‌شود.
۵. کلاینت مقدار بازگشتی را دریافت می‌کند، گویی تابعی محلی صدا زده است.

۳ تفاوت فراخوانی محلی و فراخوانی راه دور

جنبه	فراخوانی محلی	فراخوانی راه دور
سرعت	میکروثانیه	میلی ثانیه تا ثانیه
خطا	نادر و قابل پیش‌بینی	شبکه ممکن است قطع یا تأخیر داشته باشد
حافظه	مشترک بین فراخواننده و تابع	هیچ حافظه مشترکی وجود ندارد
اتمیک بودن	تضمین شده	تضمینی نیست
آرگومان	اشاره‌گر قابل پاس دادن است	باید کپی و سریالایز شود
هزینه	صفر یا بسیار کم	سربار شبکه و سریالایزیشن
وضعیت عملیات	مشخص است	ممکن است مبهم باشد (partial failure)

جدول ۱: مقایسه فراخوانی محلی و راه دور

۴ نقش Client Stub و Server Stub

۱.۴ Client Stub

کدی است که روی ماشین کلاینت اجرا می‌شود و ظاهر یک تابع معمولی را دارد. مسئولیت‌های آن عبارت‌اند از:

- دریافت آرگومان‌ها از کد فراخواننده
- سریالایز کردن آرگومان‌ها به قالب قابل انتقال در شبکه
- ارسال درخواست به سرور و انتظار برای پاسخ
- دسریالایز کردن نتیجه و بازگرداندن آن به فراخواننده

۲.۴ Server Stub

کدی که روی ماشین سرور قرار دارد. مسئولیت‌های آن عبارت‌اند از:

- گوش دادن برای درخواست‌های ورودی
- دسریالایز کردن آرگومان‌ها

□ فراخوانی تابع واقعی پیاده‌سازی شده روی سرور

□ سریالایز کردن نتیجه و ارسال پاسخ

Stub‌ها معمولاً به صورت خودکار از روی تعریف IDL تولید می‌شوند و برنامه‌نویس نیازی به نوشتن دستی آن‌ها ندارد.

۵ مفهوم Serialization و Deserialization

سریالایزیشن فرایند تبدیل ساختار داده‌ای موجود در حافظه به یک دنباله از بایت‌هاست که قابل انتقال از طریق شبکه باشد. **دسریالایزیشن** عکس این فرایند است: بازسازی ساختار داده از دنباله بایت دریافت‌شده. اهمیت این مفهوم در RPC از آن جهت است که:

□ ساختارهای داده‌ای مثل struct، آرایه، یا map نمی‌توانند مستقیماً از طریق شبکه منتقل شوند.

□ دو ماشین ممکن است معماری، endianness، یا زبان برنامه‌نویسی متفاوتی داشته باشند.

□ فرمت سریالایز شده باید بین طرفین توافق‌شده و یکسان باشد.

فرمت‌های رایج شامل Protocol Buffers (فشرده و باینری)، JSON (خوانا برای انسان)، XML (پرحجم)، و MessagePack (باینری سبک) می‌شوند.

۶ مفهوم IDL □ زبان تعریف رابط

Interface Definition Language یا IDL زبانی است که با آن رابط سرویس‌های RPC تعریف می‌شود: نام تابع‌ها، نوع آرگومان‌ها، و نوع مقدار بازگشتی. این تعریف مستقل از زبان برنامه‌نویسی است و نقش آن:

□ ایجاد قرارداد مشترک بین کلاینت و سرور

□ تولید خودکار stubها برای زبان‌های مختلف

□ مستندسازی رابط سرویس به شکل دقیق و ماشین‌خوانا

به عنوان نمونه، تعریف یک سرویس احراز هویت در gRPC با Protocol Buffers:

```
service AuthService {  
    rpc Login(LoginRequest) returns (LoginResponse);  
}  
  
message LoginRequest {  
    string username = 1;  
    string password = 2;  
}  
  
message LoginResponse {  
    bool    success = 1;  
    string message = 2;  
}
```

۷ خطاهای رایج در RPC

خطاهایی که در فراخوانی راه دور ممکن است رخ دهند ولی در فراخوانی محلی وجود ندارند:

۱. خطای شبکه: قطع ارتباط، timeout، یا از دست رفتن بسته‌ها.
۲. خطای سرور: سرور کرش کرده یا در دسترس نباشد.
۳. ابهام در وضعیت (Partial Failure): کلاینت نمی‌داند آیا سرور درخواست را دریافت و اجرا کرد یا نه.
۴. اجرای مکرر: به دلیل retry، تابع ممکن است دوبار اجرا شود (at-least-once در مقابل exactly-once).
۵. ناسازگاری نسخه: کلاینت و سرور IDL متفاوتی داشته باشند.
۶. خطای سریالایزیشن: داده‌ای که قابل سریالایز نباشد یا در مقصد درست دسریالایز نشود.
۷. تأخیر غیرقابل پیش‌بینی: برخلاف توابع محلی، زمان پاسخ ممکن است به شدت متغیر باشد و طراحی سیستم باید آن را در نظر بگیرد تا از خطاهای دیگر جلوگیری کند.

۸ مقایسه REST با RPC

REST (Representational State Transfer) یک سبک معماری برپایه HTTP است که منابع را در مرکز می‌گذارد، در حالی که RPC تمرکز روی عمل (تابع) دارد.

معیار	RPC	REST
تمرکز	عمل (تابع)	منبع (resource)
پروتکل	متنوع (TCP، HTTP، ...)	HTTP
فرمت پیام	متنوع (JSON، binary، ...)	معمولاً JSON یا XML
قرارداد	IDL اجباری	اختیاری (OpenAPI)
کارایی	بالا (به خصوص با فرمت باینری)	متوسط
خوانایی	کمتر	بیشتر (JSON خوانا)
Caching	دشوار	آسان (از HTTP cache بهره می‌برد)
Streaming	پشتیبانی مستقیم (مثل gRPC)	محدود
یادگیری	نیاز به ابزار و IDL	ساده‌تر

جدول ۲: مقایسه REST و RPC

۹ مقایسه فناوری‌های رایج RPC

۱.۹ gRPC

توسط Google توسعه یافته و از Protocol Buffers به عنوان IDL و فرمت سریالایزیشن استفاده می‌کند. روی HTTP/2 اجرا می‌شود و از streaming دوطرفه پشتیبانی می‌کند. مناسب‌ترین گزینه برای microservice‌های با کارایی بالا و ارتباطات داخلی سیستم‌های توزیع شده است.

۲.۹ JSON-RPC

یک پروتکل ساده و سبک است که پیام‌ها را در قالب JSON رد و بدل می‌کند. نیاز به IDL جداگانه ندارد و پیاده‌سازی آن بسیار ساده است. مناسب برای API‌های سبک و سرویس‌هایی که سادگی در آن‌ها اولویت دارد.

۳.۹ XML-RPC

پیشینه JSON-RPC است و از XML برای انکودینگ پیام‌ها استفاده می‌کند. به دلیل حجم بالای XML کارایی کمتری دارد اما در سیستم‌های قدیمی هنوز کاربرد دارد.

۴.۹ Apache Thrift

توسط Facebook ساخته شده و مانند gRPC دارای IDL اختصاصی است. از زبان‌های برنامه‌نویسی متعددی پشتیبانی می‌کند و در سیستم‌های داخلی شرکت‌های بزرگ کاربرد دارد.

۵.۹ Java RMI

مخصوص اکوسیستم Java است. اشیاء Java را مستقیماً سریالایز می‌کند و تنها بین برنامه‌های Java قابل استفاده است. ساده برای پروژه‌های Java-محور اما فاقد پشتیبانی چندزبانه است.

۶.۹ SOAP

پروتکلی مبتنی بر XML و HTTP که در سرویس‌های وب سازمانی استفاده می‌شود. دارای استانداردهای امنیتی و WS-* غنی است اما به دلیل پیچیدگی، امروزه کمتر استفاده می‌شود.

فناوری	فرمت داده	IDL	چندزبانه	Streaming	موارد استفاده
gRPC	Protobuf	بله	بله	بله	Microservices
JSON-RPC	JSON	خیر	بله	خیر	API های ساده
XML-RPC	XML	خیر	بله	خیر	سیستم‌های قدیمی
Thrift	Binary/JSON	بله	بله	بله	سرویس‌های داخلی
Java RMI	Java Serial.	ضمنی	خیر	خیر	پروژه‌های Java
SOAP	XML	WSDL	بله	خیر	سازمانی

جدول ۳: جدول مقایسه فناوری‌های RPC

۱۰ پرسش‌های تحلیلی

۱.۱۰ چرا RPC فراخوانی راه دور را شبیه فراخوانی محلی می‌کند؟

RPC با استفاده از stub این انتزاع را ایجاد می‌کند. Client stub دقیقاً همان امضای تابع واقعی را دارد؛ پس برنامه‌نویس کد خود را مثل همیشه می‌نویسد. تمام منطق شبکه، سریالایزیشن، و ارتباط در لایه stub پنهان است و کد فراخواننده هیچ چیزی از آن نمی‌بیند. این اصل به Location Transparency معروف است.

۲.۱۰ چرا این شباهت می‌تواند خطرناک یا گمراه‌کننده باشد؟

شباهت ظاهری ممکن است برنامه‌نویس را از تفاوت‌های بنیادی غافل کند:

- تابع محلی هرگز به دلیل «قطعی شبکه» شکست نمی‌خورد؛ فراخوانی راه دور ممکن است.
- ممکن است نتوانیم بدانیم آیا درخواست اجرا شد یا نه (partial failure).
- اگر برنامه‌نویس retry اضافه کند و تابع idempotent نباشد، عمل ممکن است دوبار انجام شود.
- تأخیر زمانی کاملاً متفاوت است و نادیده گرفتن آن به مشکلات جدی کارایی منجر می‌شود.
- مدیریت خطا در RPC پیچیده‌تر است چون باید حالت‌های بیشتری را پوشش دهد.

۳.۱۰ چه خطاهایی در فراخوانی راه دور ممکن است رخ دهد که در فراخوانی محلی وجود ندارند؟

- Network timeout: پاسخ در زمان مقرر نرسید.
- Connection refused: سرور در دسترس نیست یا کرش کرده است.
- شکست جزئی (Partial Failure): سرور درخواست را دریافت و اجرا کرد اما پاسخ در شبکه گم شد.
- ناسازگاری نسخه: کلاینت و سرور با IDL‌های متفاوت کار می‌کنند و فیلدی وجود دارد که یک طرف آن را نمی‌شناسد.
- خطای سریالایزیشن: داده در مقصد قابل بازسازی نیست.
- تأخیر متغیر (Jitter): زمان پاسخ‌دهی به دلیل شرایط شبکه قابل پیش‌بینی نیست.

۴.۱۰ در چه شرایطی gRPC انتخاب بهتری از REST است؟

- وقتی کارایی و تأخیر کم اهمیت زیادی دارد (ارتباط داخلی microservice‌ها).
- وقتی به streaming دوطرفه یا یک‌طرفه نیاز است.
- وقتی قرارداد قوی بین تیم‌ها لازم است و IDL اجباری مزیت است.
- وقتی از چند زبان برنامه‌نویسی مختلف استفاده می‌شود و کد stub باید خودکار تولید شود.

□ وقتی حجم داده زیاد است و فرمت باینری فشرده اهمیت دارد.

۵.۱۰ در چه شرایطی REST انتخاب ساده‌تر یا مناسب‌تر است؟

□ وقتی API قرار است عمومی باشد و توسط کلاینت‌های ناشناس مصرف شود.

□ وقتی خوانایی و قابلیت debug راحت اهمیت دارد (پیام‌ها JSON خوانا هستند و با curl قابل تست‌اند).

□ وقتی از caching استاندارد HTTP بهره‌مند می‌شویم.

□ وقتی تیم آشنایی کمتری با ابزارهای gRPC دارد و سرعت توسعه اولویت است.

□ وقتی طرف مقابل یک مرورگر یا JavaScript کلاینت است که REST را راحت‌تر مصرف می‌کند.